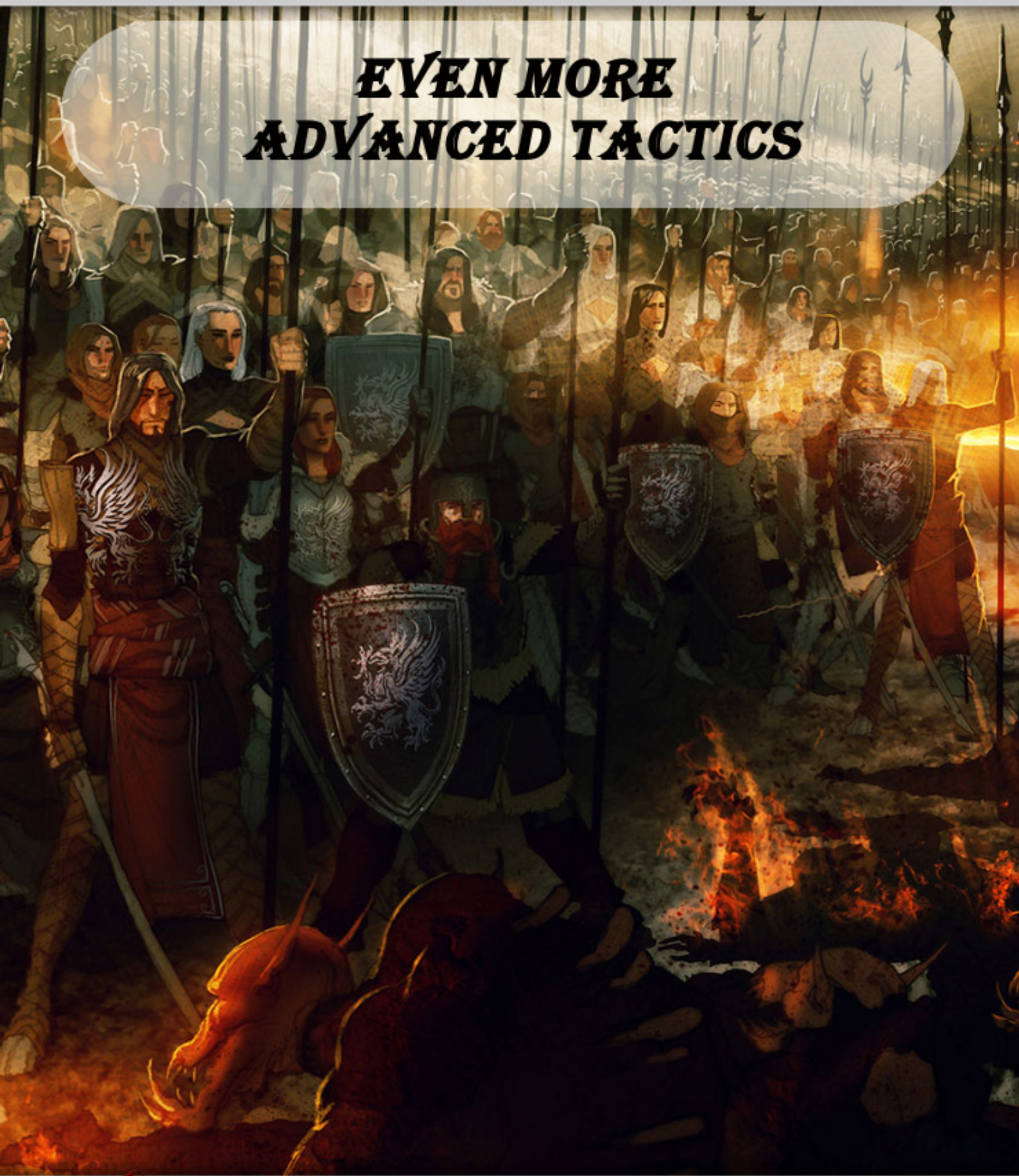


***EVEN MORE
ADVANCED TACTICS***





Contents

1	Introduction	7
1.1	Thanks and Remarks	7
1.2	About (Even More) Advanced Tactics	7
2	Installation	9
2.1	Installation	9
2.1.1	Main components	9
2.1.2	Optional components	9
2.2	Compatibility	9
2.2.1	Events	9
2.2.2	Strings	9
3	Definitions	11
3.1	Tactic Table	11
3.2	Tactic (aka Tactic Row)	11
3.3	Command	11
3.4	Party	12
3.5	Enemies	12
4	Custom AI Flow	15
4.1	Game Engine and AI Script	15
4.2	Behavior and Tactic Table	15
4.3	*Special Commands	16

5	Behaviors	17
5.1	Overview	17
5.2	Components of Behavior	17
5.2.1	Avoid Area of Effect	17
5.2.2	Switch Weapon	18
5.2.3	Avoid Enemies	18
5.2.4	Attack	18
6	Commands	21
6.1	Break	21
6.2	Deactivate all stances	21
6.3	Move at short/Move at medium	21
6.4	*Set as Target	21
6.5	*Store as Saved Friend / Enemy	22
7	Target Types	23
7.1	Self	23
7.2	Enemy	23
7.3	Team Member	23
7.4	Party Member	23
7.5	Saved Enemy	24
7.6	Saved Friend	24
7.7	Target	24
7.8	Hero	24
7.9	Controlled Party Member	24
8	Ability Mutex	25
8.1	Overview	25
8.2	Ability Mutex Categories	25
9	MultiCondition	27
9.1	Overview	27
9.2	MultiCondition Syntax Rules	27
9.3	MultiCondition Settings	28
9.4	MultiCondition Enemy	29
9.5	MultiCondition Friend	29
10	Boolean Variables	33
10.1	Overview	33

10.2	Rules for Boolean Variables	33
11	FAQ	35
	Index	37



1. Introduction

1.1 Thanks and Remarks

- People in Bioware, for creating this great game
- anakin55, for creating his marvelous mod - *Advanced Tactics*.
- tinman, for his implementation of *Clustered with X enemies*. It saved me a lot of time.
- Talmud, for his *Unlimited Variable Storage*
- hfuet, for extensive tests of this mod and suggestions.
- HappyDog, for being happy.
- kajana, for testing.
- DeJuan N. Onley, for testing.
- zalunaya, for map images used in this manual

1.2 About (Even More) Advanced Tactics

It all started with anakin55 and his marvelous mod - *Advanced Tactics*. Before him, there was only darkness. Bioware had a great idea to allow players to create their own AI using Tactics. There was one problem- default AI was so stupid that you had to put all your effort to keep your characters from killing themselves. *Advanced Tactics* by anakin55 changed all this. It was amazing to play DAO with this mod. However, during my gameplay I have noticed that it lacks a lot of features that, from my point of view, seemed obvious for such a mod. Fortunately, anakin55 was so kind to publish his source code so I could add new features, one by one. Some time ago I realized that my bunch of modifications have grown up to the size of full mod. I started thinking: 'It may be a good idea to share it with community'. It turned out to be much more time consuming then I expected. It took me several months to optimize the code, rewrite it and test it, but here we are - *Even More Advanced Tactics (EMAT)* is released.

Have fun,

bratrahen (aka MkBot)

Installation

Main components
Optional components

Compatibility

Events
Strings

Deep Roads

To Frostback
Mountains

2. Installation

2.1 Installation

2.1.1 Main components

1. Uninstall or disable *Advanced Tactics* by anakin55
2. Launch *daupdater.exe* (located in `.../DragonAge/bin_ship/`) and install *EMAT*

2.1.2 Optional components

- In order to increase number of tactics to 50 put *exptable.gda* and *exptable_gxa.gda* files in the following location: `.../BioWare/DragonAge/packages/core/override/`

2.2 Compatibility

You must rewrite your Tactics after installing *EMAT*.

2.2.1 Events

This add-in uses the *Event Manager* by anakin55.

EMAT overrides two events:

- `EVENT_TYPE_COMMAND_COMPLETE`
- `EVENT_TYPE_PERCEPTION_APPEAR`

EMAT listens for three events:

- `EVENT_TYPE_COMMAND_PENDING`
- `EVENT_TYPE_DYING`
- `EVENT_TYPE_PARTY_MEMBER_HIRED`

EMAT creates one new event:

- `AT_EVENT_TYPE_SKILL_OPTIONS` (ID = 500)

2.2.2 Strings

- strings IDs from 8000000 to 8000132 (inherited from *Advanced Tactics*)
- strings IDs from 9000000 to 9000700



3. Definitions

3.1 Tactic Table

Tactic Look at Figure 3.1. *Target Type* + *Condition* + *Command*

Tactic Table Look at Figure 3.1

Tactic Owner The owner of *Tactic Table*. Look at Figure 3.1. Here Alicja is a *Tactic Owner*



Figure 3.1: *Tactic* and *Tactic Table*

3.2 Tactic (aka Tactic Row)

Tactic Id Look at Figure 3.2

Target Type Look at Figure 3.2

BaseCondition Look at Figure 3.2

SubCondition Look at Figure 3.2

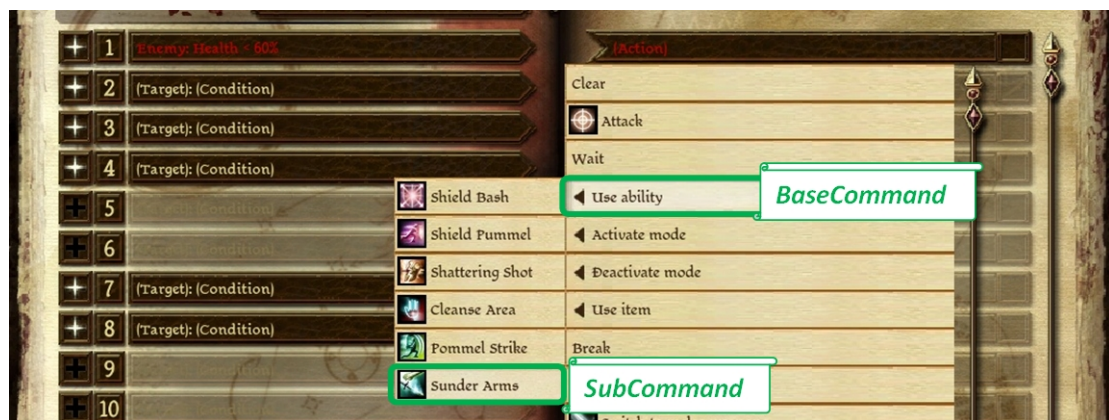
Condition *BaseCondition* + *SubCondition*

3.3 Command

BaseCommand Look at Figure 3.3

SubCommand Look at Figure 3.3

Command *BaseCommand* + *SubCommand*

Figure 3.2: *Tactic Id, Target Type and Condition*Figure 3.3: *Command*

3.4 Party

Party Pool Look at Figure 3.4

Team Members Look at Figure 3.4 and Figure 3.5

Party Members *Team Members* + *Summoned*. Look at Figure 3.5

Summoned Look at Figure 3.5

3.5 Enemies

Current Target Look at Figure 3.6

Creatures List Each *Tactic* is composed of three parts: *Target Type*, *Condition* and *Command*.

AI chooses one creature of given *Target Type* that satisfies *Condition* and then *Command* is executed on it. How does AI choose a creature? It performs check on each creature until it finds one that satisfies *Condition* (or it turns out that there is no such a creature). Simple? Yes, but there is a catch. Notice, **FIRST** creature that satisfies *Condition* is returned! Thus, it is important how creatures are sorted (by distance? by health? by mana? etc.). *Creatures List* is a sorted list of creatures. The way it is sorted depends on chosen *Target Type* (in description of each *Target Type* there is info how *Creatures List* is sorted). Look at Figure 3.7 for an example of *Creatures List* for *Target Type:Enemy*. Alice has *Current Target* set at Genlock 2. *Target Type:Enemy* prefers *Current Target* so Genlock 2 is put at the beginning of *Creatures List*. Other enemies are sorted by their proximity to Alice.



Figure 3.4: *Party Pool* and *Team Members*



Figure 3.5: *Team Members*, *Summoned* and *Party Members*



Figure 3.6: *Current Target*



Figure 3.7: *Creatures List (Enemies)* - *Current Target* is set at Genlock 2

4. Custom AI Flow

4.1 Game Engine and AI Script

There are two things that control AI, each with defined responsibility:

- *Game Engine* - it is responsible for executing commands
- *EMAT* Script - it is responsible for choosing next command to execute

Tactics system gives player control over *EMAT* Script part. You can define your own *Tactic Table* and choose a *Behavior* to decide which *Command* should be executed. On the other hand there is absolutely no control over *Game Engine* so once the order is issued there is nothing *EMAT* Script can do until *Command* is finished.

The interaction between *EMAT* Script and *Game Engine* is as follows (simplifying of course):

1. At the beginning, character stands still and does nothing, there is no *Command* to execute.
2. Game Engine asks *EMAT* Script what the next command should be.
3. *EMAT* evaluates *Tactics* and chooses next *Command*
4. Game Engine executes given *Command*
5. Game Engine acknowledges *EMAT* Script whether *Command* finished successfully or not.
6. Game Engine asks *EMAT* Script for new *Command*
7. Repeat...

4.2 Behavior and Tactic Table

AI of each character is composed of two parts:

- *Behavior*
- *Tactic Table*

Behavior is a set of predefined actions. In order to use *Behavior* effectively you have to know when it is executed. Before *Tactic Table*? After? In fact, *Behavior* consists of several components and some of them are executed before and some are executed after *Tactic Table*. For detailed description of *Behavior*'s components see next chapter. Here we outline AI flow using as an example Alice the Spirit Healer.

■ **Example 4.1** Alice is a Spirit Healer and *Tactic Table* is extremely short. There are just three *Tactics* visible (Figure 4.1) but under the hood there is much more. Look at Figure 4.2 - there are additional hidden *Tactics* that handle *Behavior*'s components. ■

4.3 *Special Commands

Most *Commands* are used to issue a specific order to character (eg. *Attack* or *Use ability*) but some of them are used to give a Player more control over AI (eg. **Store as Saved Enemy*). These commands are marked with star [*] and they are not returned to the Game Engine. Instead, *EMAT* Script executes them itself and then immediately evaluates next *Tactic* in *Tactic Table*. Look at Figure 6.1 for usage example.

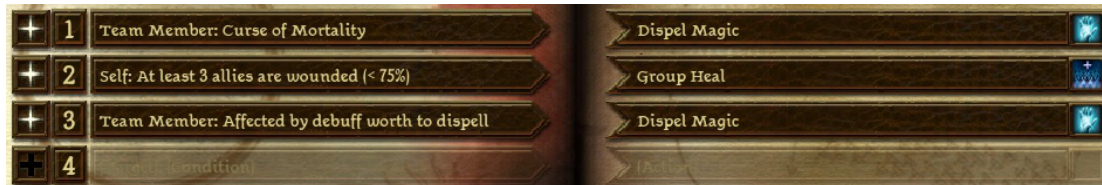


Figure 4.1: There are only three *Tactics* visible...

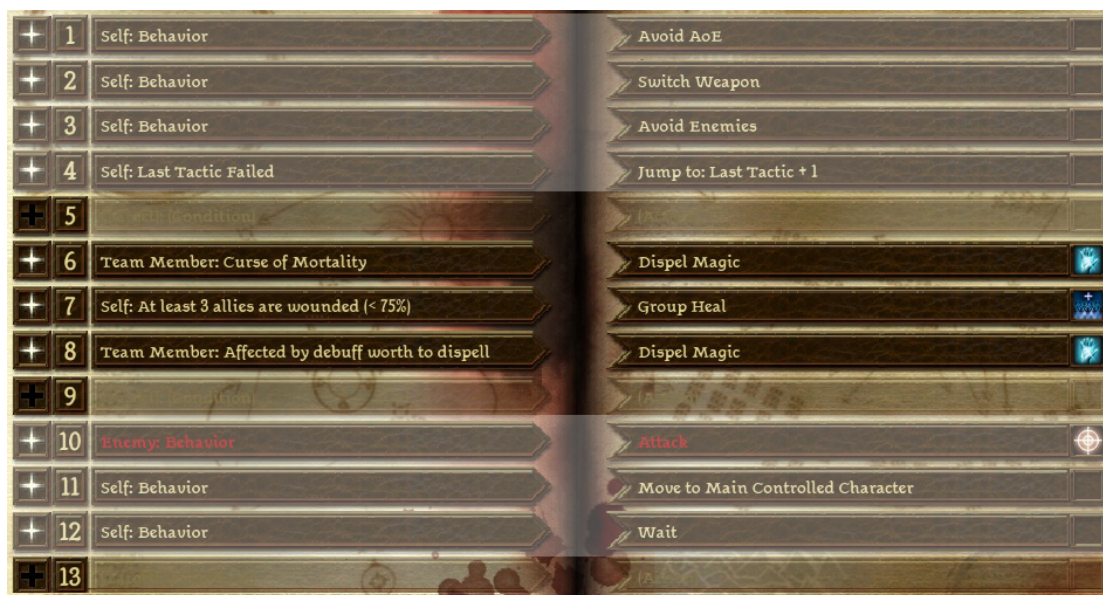


Figure 4.2: ... but in fact there are always hidden *Tactics* added by *Behavior*

Overview

Components of Behavior

Avoid Area of Effect
Switch Weapon
Avoid Enemies
Attack

5. Behaviors

5.1 Overview

In contrary to popular believe, *Behavior* is not just a default action executed if all *Condition* in *Tactic Table* fails. In fact, according to previous chapter, there are four components of *Behavior*, and some of these components are executed even before *Tactic Table*. An overview of these components for each *Behavior* is presented below and in Table 5.1.

Default *Default* uses melee weapon then attacks enemies in party proximity. If uses ranged weapon then attacks enemies in line of sight.

Passive *Passive* neither switches weapon nor attacks enemies. It is intended for characters that are fully controlled by *Tactic Table*.

Aggressive *Aggressive* is intended for characters that charge toward enemies. *Aggressive* attacks and looks for enemies on much greater area than *Default*.

Ranged *Ranged* is intended for archers. It strongly prefers ranged weapon, tries to avoid Nearby Enemies and doesn't attack in melee.

Cautious *Cautious* is intended for support characters. Attack on current target is continued but it doesn't choose a new one. *Cautious* doesn't want to draw opponents attention.

Defensive *Defensive* prefers to attack from distance but he doesn't escape from enemies. *Defensive* switches to melee and engages opponents if they are close enough.

Protective *Protective* is intended for fighters that use bow but melee weapon is their primary weapon of choice. This behavior switches to bow when there is no enemies in proximity so fighter can weaken opponents before clashing in melee. Fighter immediately switches to melee and attacks any enemy if he come too close to any *Team Member*. This way *Protective* can efficiently protect support characters.

Templar *Templar* is like *Default* behavior but has *Avoid AoE* feature disabled. It is intended for characters with high magic/elemental resistance and/or using [Indomitable]/[Shield Wall]

5.2 Components of Behavior

5.2.1 Avoid Area of Effect

Feature rewritten by anakin55 in his *Advanced Tactics*. All creatures (including enemies!) tries to leave area of effect of active or incoming hostile spells (e.g. Blizzard or Fireball). This happens before *Tactic Table* is evaluated.

Behavior	Avoid AoE	Switch Weapon	Avoid Enemies	Attack	Move
Default	Yes	No	No	Yes	Yes
Passive	No	No	No	No	Yes
Aggressive	Yes	No	No	Yes	Yes
Ranged	Yes	Yes	Yes	Yes	Yes
Cautious	Yes	No	No	Yes	Yes
Defensive	Yes	Yes	No	Yes	Yes
Protective	Yes	Yes	No	Yes	Yes
Templar	No	No	No	Yes	Yes
Default Stationary	Yes	No	No	Yes	No
Passive Stationary	No	No	No	No	No
Aggressive Stationary	Yes	No	No	Yes	No
Ranged Stationary	Yes	Yes	Yes	Yes	No
Cautious Stationary	Yes	No	No	Yes	No
Defensive Stationary	Yes	Yes	No	Yes	No
Protective Stationary	Yes	Yes	No	Yes	No
Templar Stationary	No	No	No	Yes	No

Table 5.1: Behaviors: Overview

5.2.2 Switch Weapon

Feature rewritten in *EMAT*. Character switches his weapon set to melee/ranged one. This happens before *Tactic Table* is evaluated. Look at Table 5.2 for more details.

Behavior	Melee Weapon	Ranged Weapon
Default	No	No
Passive	No	No
Aggressive	No	No
Ranged	No	No Enemies within Short range of Self
Cautious	No	No
Defensive	An Enemy within 2xShort range of Self	No Enemies within Medium range of Self
Protective	An Enemy within 2xShort range of Any Team Member	No Enemies within Medium range of all Team Members
Templar	No	No

Table 5.2: Behaviors: Switch Weapon

5.2.3 Avoid Enemies

Feature rewritten by anakin55 in *Advanced Tactics*. Character runs away from nearest enemy but he never goes too far from *Controlled Character*.

5.2.4 Attack

Feature rewritten in *EMAT*. Behaviors prefer to attack *Current Target* but limitations of given *Behavior* still applies. In case of using melee weapon, *Enemy* must be within specified attack

range e.g. *Aggressive* attacks *Current Target* only if it is within $[2 \times \text{Medium}]$ range of any *Team Member*. In case of using ranged weapon, *Current Target* must be in line of sight. Otherwise, *Behavior* looks for a new target. Look at Table 5.3 for more details.

Behavior	Melee Attack	Ranged Attack
Default	Enemy within Medium range of any Team Member. Prefers Current Target	Enemy in line of sight. Prefers Current Target.
Passive	No	No
Aggressive	Enemy within $2 \times \text{Medium}$ range of any Party Member. Prefers Current Target	Enemy in line of sight. Prefers Current Target
Ranged	No	Enemy in line of sight. Prefers Current Target
Cautious	Current Target if within $2 \times \text{Short}$ range of Self	Enemy in line of sight. Prefers Current Target
Defensive	Enemy within Medium range of Self. Prefers Current Target	Enemy in line of sight. Prefers Current Target
Protective	Enemy within Medium range of any Team Member. Prefers Current Target	Enemy in line of sight. Prefers Current Target
Templar	Enemy within Medium range of any Team Member. Prefers Current Target	Enemy in line of sight. Prefers Current Target

Table 5.3: Behaviors: Attack



Break
Deactivate all stances
Move at short/Move at medium
*Set as Target
*Store as Saved Friend / Enemy

6. Commands

I do not intend to describe all commands in this chapter, it would be quite boring. Only those that are not self-explaining are listed here.

6.1 Break

It is similar to *Jump To* but it does not jump to specific *Tactic*. Instead, it jumps over all *Tactics*, straight to *Behavior*.

6.2 Deactivate all stances

This *Command* deactivates **almost** all *Stances* (aka *Modal Abilities*). This makes deactivation of your *Stances* after combat much easier. As I said, almost all *Stances* are deactivated. Below is a list of those, that are not:

- Animate Dead
- Summon Wolf
- Summon Bear
- Summon Spider
- Indomitable (It has quite long cooldown. Turned out it is better to leave Indomitable activated.)

6.3 Move at short/Move at medium

Moves *Tactic Owner* to given friendly character. If *Tactic Owner* is within defined distance to his target (*Short/Medium*) then this *Command* fails because we do not have to go anywhere. Use this *Command* to move your characters around.

6.4 *Set as Target

This *Command* changes your *Current Target*. If there is no creature that satisfies *Condition* then * *Set as Target* has no effect. This is implemented like ability so you will see 'Set as target' message above *Tactic Owner*'s head if you have 'Show floating spell and talent names' option enabled. Notice, this is *Special Command*.

6.5 *Store as Saved Friend / Enemy

Creature found by *Condition* is stored in *Saved Friend/Saved Enemy*. If there is no creature that satisfies *Condition* then *Saved Friend/Saved Enemy* is cleared. Notice, this is *Special Command*. Look at Figure 6.1.

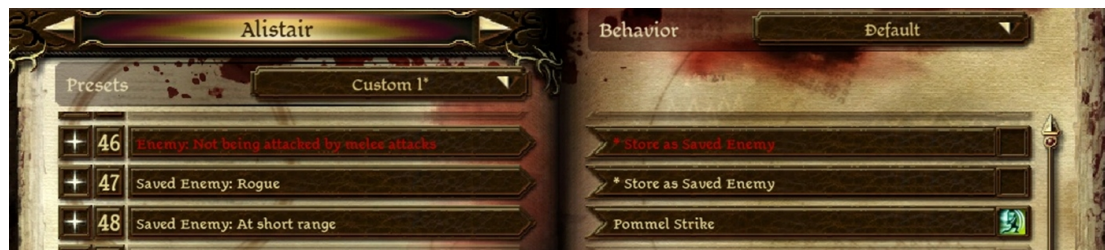


Figure 6.1: This *Tactic Table* should be read as follows: nearest *Enemy Not being attacked by melee attack* is stored in *Saved Enemy*. If *Enemy* is not a *Rogue* then *Saved Enemy* is cleared. *Pommel Strike* is used only if this *Enemy* has *Rogue* class and it is at *Short* range.



Self
Enemy
Team Member
Party Member
Saved Enemy
Saved Friend
Target
Hero
Controlled Party Member

7. Target Types

7.1 Self

Ignore Confused Confused creature (effect caused by *Waking Nightmare* spell) cannot target *Self*. The purpose of this limitation is to prevent dispelling this effect too easily.

7.2 Enemy

Any creature you can attack.

Sorting of Creatures List *Target Type Enemy* prefers *Current Target* (ie. it is tested first). Other hostile creatures are sorted by their proximity to *Tactic Owner*.

Ignore Sleep and Pinning Shot By default, *Target Type Enemy* ignores creatures that are sleeping or pinned (effects caused by *Sleep* and *Pinning Shot* ability respectively). This feature was introduced because these effects are removed once creature is attacked. If you want to target a sleeping or pinned *Enemy* (e.g. in order to use *Spell Combination Sleep + Horror*) then use *Enemy:AI Status:Sleep* or *Enemy:AI Status:Root*

Ignore Dead By default, dead creatures are ignored. If you want to target a dead *Enemy* then use *Enemy:AI Status:Dead*

7.3 Team Member

See definition of *Team Member*.

Sorting of Creatures List *Target Type Team Members* prefers *Tactic Owner* (ie. it is tested first). Other *Team Members* are sorted by their proximity to *Tactic Owner*.

Ignore Dead By default, dead creatures are ignored. If you want to target a dead *Team Member* then use *Team Member:AI Status:Dead*

Ignore Summoned *Summoned* creatures are not a *Team Members*.

7.4 Party Member

See definition of *Party Member*.

Sorting of Creatures List *Target Type Party Members* prefers *Tactic Owner* (i.e. it is tested first). Other *Party Members* are sorted by their proximity to *Tactic Owner*.

Ignore Dead By default, dead creatures are ignored. If you want to target a dead *Party Member* then use *Party Member:AI Status:Dead*

7.5 Saved Enemy

Enemy that was stored using **Store as Saved Enemy Command*. 0

Combat Start At the beginning of combat *Saved Enemy* is empty.

Tactic fails if Saved Enemy is empty If *Saved Enemy* is empty then *Condition* always fails.

Ignore Sleep and Pinning Shot By default, *Target Type Saved Enemy* ignores creature that is sleeping or pinned (effects caused by *Sleep* and *Pinning Shot* ability respectively). This feature was introduced because these effects are removed once creature is attacked. If you want to target a sleeping or pinned *Saved Enemy* eg. for *Spell Combination Sleep + Horror*) use *Saved Enemy:AI Status:Sleep*

Ignore Dead By default, dead creature is ignored. If you want to target a dead *Saved Enemy* then use *Saved Enemy:AI Status:Dead*

7.6 Saved Friend

Party Member that was stored using **Store as Saved Friend Command*.

Combat Start At the beginning of combat *Saved Friend* is empty.

Tactic fails if Saved Friend is empty If *Saved Friend* is empty then *Condition* always fails.

Ignore Dead By default, dead creature is ignored. If you want to target a dead *Saved Friend* then use *Saved Friend:AI Status:Dead*

7.7 Target

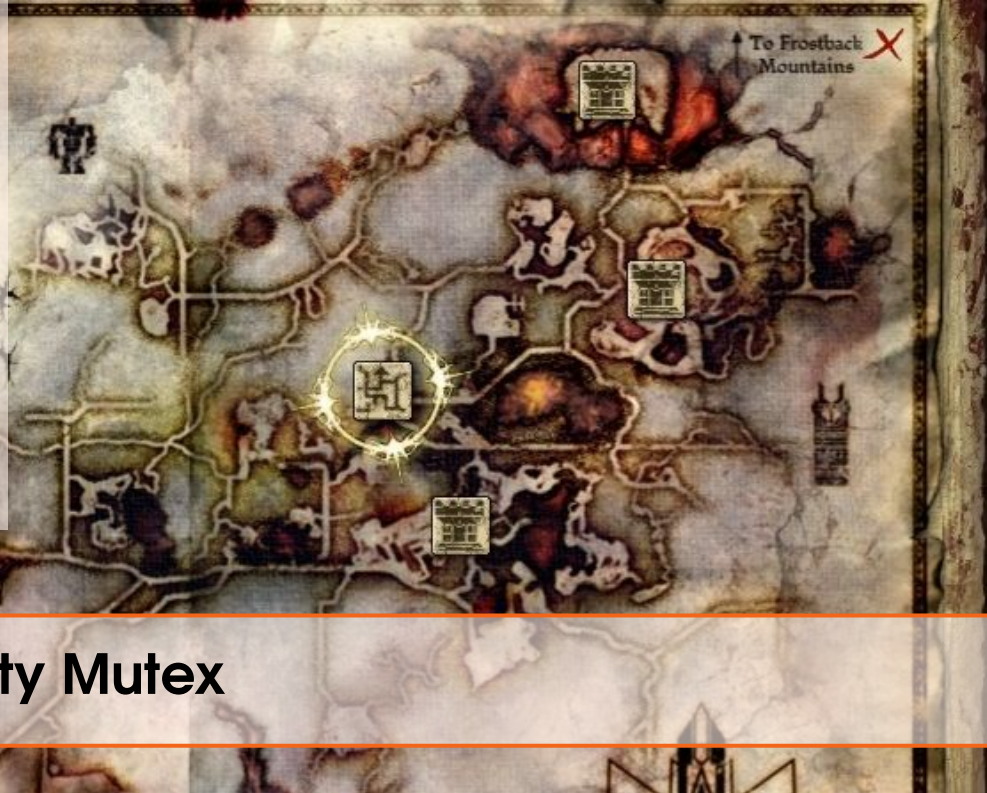
See definition of *Current Target*.

7.8 Hero

Main protagonist created by Player.

7.9 Controlled Party Member

Creature under Player's direct control.



8. Ability Mutex

8.1 Overview

Each time *Tactic* contains *Use ability* command, in addition to testing *Condition*, creatures are tested for their validity for given ability eg. if creature cannot attack then there is no sense in using *Shield Bash* on it. This works pretty well and prevents your characters from wasting all their knock/stun/etc. abilities on the same target at one instant. Unfortunately, there are situation when it is not enough. Let assume both Hero and Alistair fight using shield. Thus, their *Tactic Tables* are very similar. It is very likely that at some point both Hero and Alistair will decide that it is good idea to use *Shield Pummel* on their *Current Targets*. Notice, if they attack the same enemy then one of them will waste his ability. The point is, they were making decisions almost at the same time so they both thought that enemy is valid for *Shield Pummel*. EMAT introduces additional protection against such scenarios and it is called *Ability Mutex*. Once a character decides to use ability on a creature he sends a message to other *Party Members*. This is signaled by 'I go first!' message appearing above character's head. As result, *Party Members* will postpone using abilities with similar effects on that creature. Once ability is used it is signaled by 'Free to go!' and other *Party Members* are free to use their abilities. *Party Member* that postpones using his ability signals it by 'After you' message above his head.

8.2 Ability Mutex Categories

- **Massive Damage:** abilities that instantly kill target block using any other abilities on targeted creature
- **Can't Attack** eg. *Shield Bash*: abilities that make enemy unable to attack block other such abilities on targeted creature
- **Can't Attack AoE** eg. *Mind Blast*: abilities that make all enemies within Area of Effect unable to attack block all abilities belonging to *Can't Attack* or termCan't Attack AoE category
- **Penalty** eg. *Vulnerability Hex*: abilities that impose non-stacking penalties block the same ability on targeted creature
- **Buff** eg. *Regeneration*: abilities that grant non-stacking buff block the same ability on targeted creature
- **Dispel** eg. *Dispel Magic*: abilities that dispel spells block using spell on targeted creature



Figure 8.1: Wynne is using *Ability Mutex* to prevent wasting *Regeneration* spell

- *Dispel AoE* eg. *Anti-Magic Burst*: abilities that dispel all spells within Area of Effect block all abilities belonging to *Magic* or termDispel AoE category

9. MultiCondition

9.1 Overview

Instead of looking for a creature that satisfies a specific *Condition* (e.g. *Rank is Elite*) you can use *MultiCondition* to look for one that satisfies a whole set of *Conditions* (e.g. *Rank is Elite AND Class is Mage*). *MultiCondition* is similar to *Use current condition for next tactic* command from *Dragon Age 2*, but it is much, much more powerful.

Unfortunately, I have very limited control over GUI so this feature is not as user-friendly as I would like it to be. Thus, using *MultiCondition* requires applying *Syntax Rules* that I will explain below. Do not worry too much, those rules are quite simple and if you do any mistake then appropriate error message will appear above your *Tactic Owner's* head (Look at Figure 9.1).

As you can see on Figure 9.4, there are three special *Target Types* for *MultiCondition*:

- *Multicondition Settings*
- *Multicondition Enemy*
- *Multicondition Friend*

We will discuss them in details later. First, look at *MultiCondition Syntax Rules*.

9.2 MultiCondition Syntax Rules

Multicondition must form a single block of Tactics We will call it *MultiCondition Block*. Look at Figure 9.2

MultiCondition Block must be ended with *Store as Saved Enemy or *Store as Saved Friend
Look at Figure 9.3

MultiCondition Settings must be put at the beginning of MultiCondition Block *MultiCondition Settings* are optional but if you use them, then they must be put at the beginning of *MultiCondition Block*. Look at Figures 9.4, 9.5, 9.6, 9.7, 9.8

When using MultiCondition Settings keep the same order as in GUI *Base Conditions* available in *MultiCondition Settings* are optional but, if you use them, they must be used in the same order as they are presented in GUI. Look at Figures 9.4, 9.5, 9.6, 9.7, 9.8

Do not mix Sorting Preferences Friend and Sorting Preferences Enemies Choose one of them, appropriate for *Target Type* you intend to search for. This is GUI limitation that they are both visible at the same time.

Do not mix MultiCondition Friend and MultiCondition Enemy Also, if you are using sorting

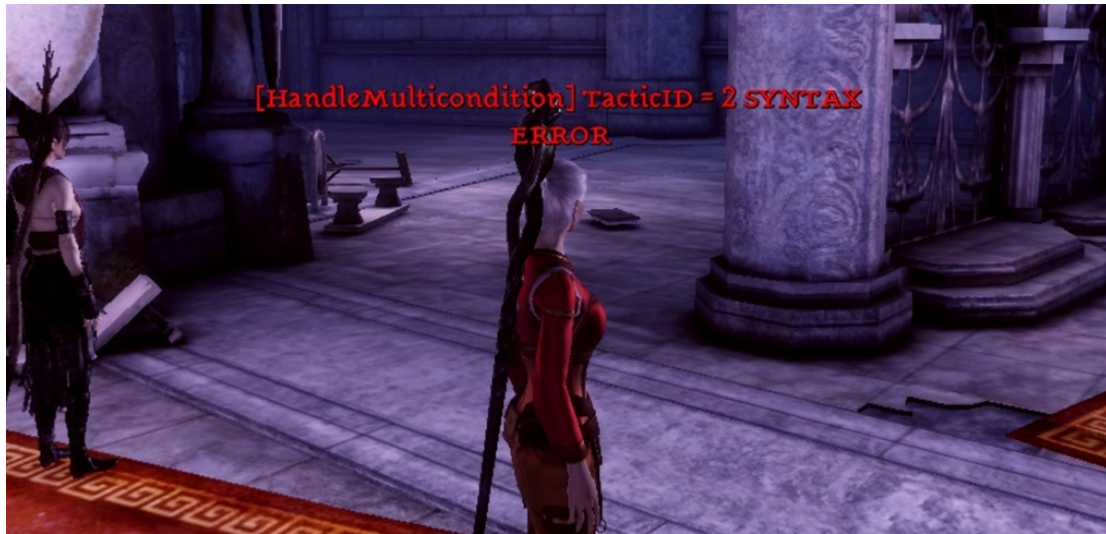


Figure 9.1: Message signaling *MultiCondition Syntax Error*. Notice that *Tactic Id* is provided to make finding error easier.

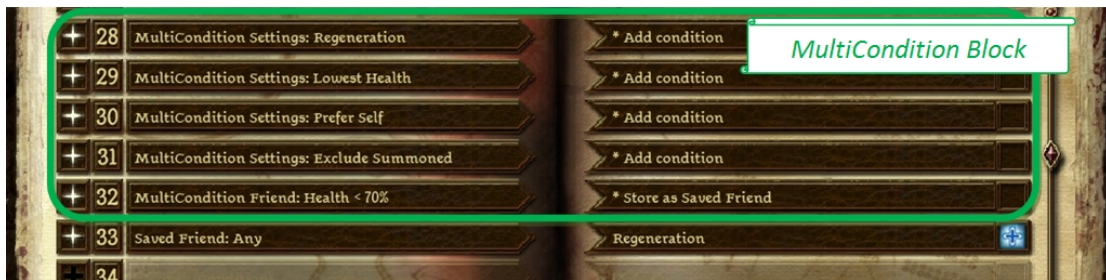


Figure 9.2: Example of *MultiCondition* usage with marked *MultiCondition Block*. Similar result could be achieved much easier but this is just an example to show how to use *MultiCondition*.

preferences, you must use appropriate one:

- for *MultiCondition Friend* use *Sorting Preferences Friend*
- for *MultiCondition Enemy* use *Sorting Preferences Enemy*

9.3 MultiCondition Settings

If you want to use any *MultiCondition Settings* then you must put them at the beginning of a *MultiCondition Block*. There are four special *Base Conditions* available in *MultiCondition Settings*. If you want to use more than one then they must appear in exactly the same order as they are listed in GUI (i.e. first *Is Valid for Ability*, then *Sort By*, then *Preferences Friend* or *Preferences Enemy*)

Is Valid for Ability This is the same check that is performed when you use *Use ability Command* (e.g. *Enemy:Any:Use Ability:Shield Bash*). The difference is, you can specify more than one *Ability* and it suffice that a creature is valid for at least one of them (in formal terms this is OR condition). Look at Figure 9.4

Sort by Determines how *Creatures List* is sorted. You can decide to sort it by lowest/highest health, rank etc. You can add more than one *Sort by* clause, they are applied if creatures happen to be 'equal' (eg. when you sort by *Rank* then all with Normal Rank are equal). Look at Figure 9.5.

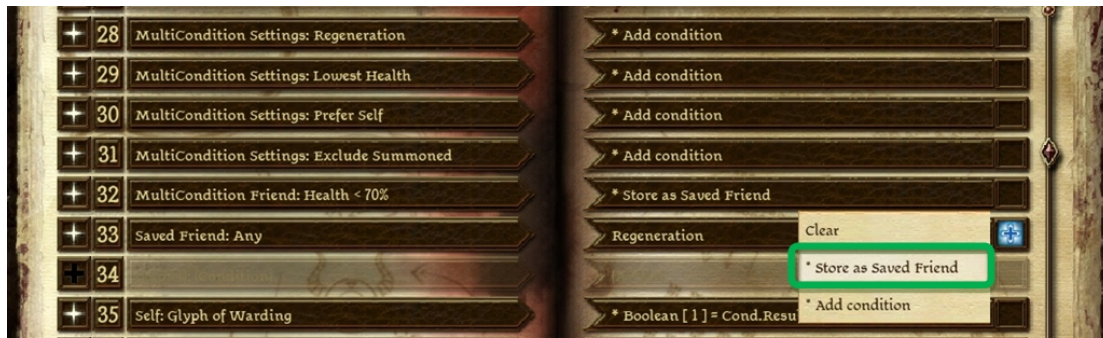


Figure 9.3: *MultiCondition* must always be ended with **Store as Saved Friend* or **Store as Saved Enemy*

Preferences Enemy/Friend Preferences modifies the way *Sort by* works. You can specify certain creatures that you want to be exactly at given position of *Creatures List* (e.g. you want your *Current Target* to be checked first as it is for *Target Type Enemy*). You can add more than one preferences, in such a case the order you defined them is taken into account. You can also exclude some creatures from *Creatures List* eg. using *Exclude Summoned*. Look Figures 19 and 20

9.4 MultiCondition Enemy

BaseConditions available for *MultiCondition Enemy* work exactly the same way as they work for *Enemy*. You can add more than one *Condition* - creature must satisfies them all.

By default, *Creatures List* is sorted by creature's proximity to *Tactic Owner* You can use *MultiCondition Settings:Sort by* to change it.

By default, dead creatures are ignored You can use *MultiCondition Settings:Preferences Enemy:Include Dead* to change it.

9.5 MultiCondition Friend

BaseConditions available for *MultiCondition Friend* work exactly the same way as they work for *Team Member* and *Party Member*. You can add more than one *Condition* - creature must satisfies them all.

By default, *Creatures List* is sorted by creature's proximity to *Tactic Owner* You can use *MultiCondition Settings:Sort by* to change it.

By default, dead creatures are ignored You can use *MultiCondition Settings:Preferences Friend:Include Dead* to change it.



Figure 9.4: There are three special *Target Types* for *MultiCondition*. If we want to use *MultiCondition Settings* (it is optional) then it must go first. In our example we want our target to be valid for *Regeneration* spell. *Regeneration* cannot be cast on character affected by *Curse of Mortality*



Figure 9.5: We want to cast *Regeneration* on the most wounded character.



Figure 9.6: It is wise to heal yourself first. If healer is dead then the game is over

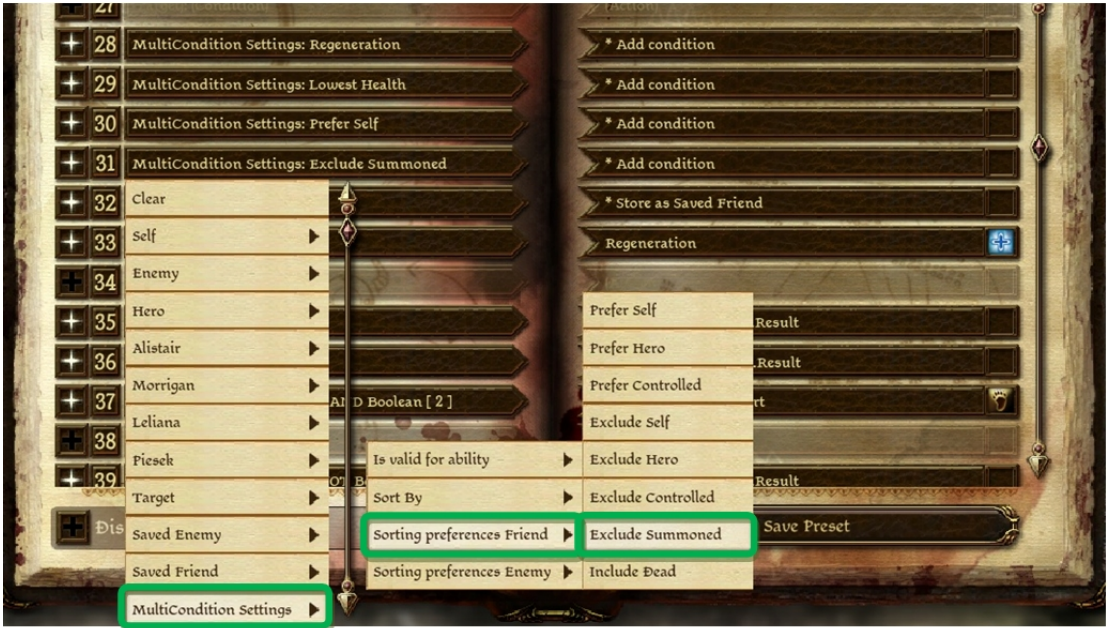


Figure 9.7: We do not want to waste mana to heal *Summoned*

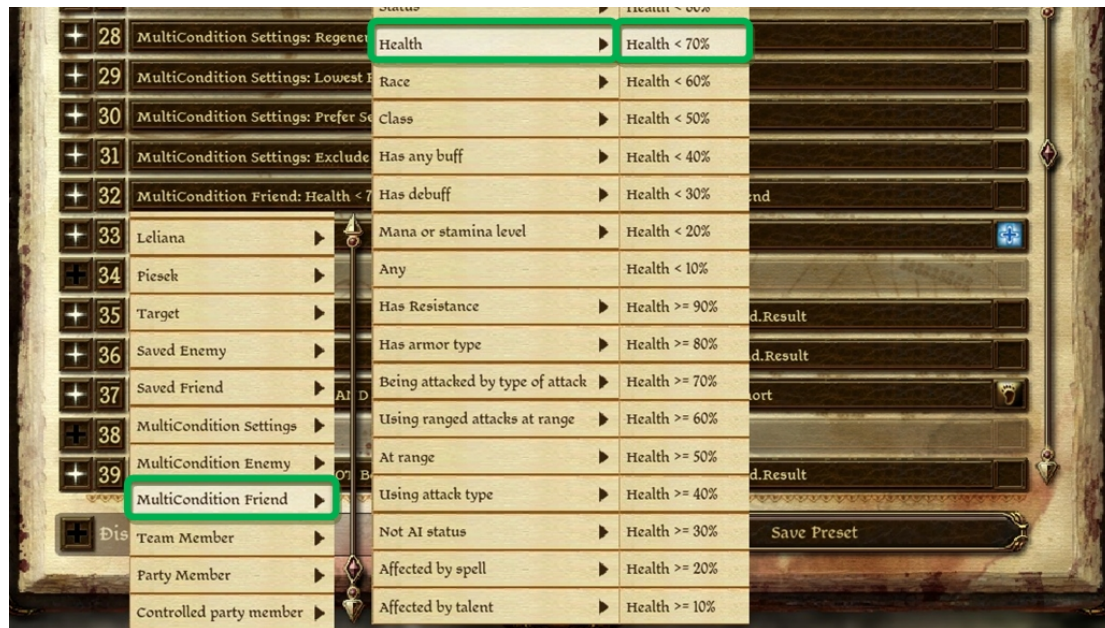


Figure 9.8: We want to cast *Regeneration* only on those characters that are really wounded. Remember that *MultiCondition Friend* must be put after *MultiCondition Settings*

10. Boolean Variables

10.1 Overview

EMAT gives for your disposal two *Boolean Variables* (ie. you can store TRUE or FALSE value there) and several *Conditions* and *Commands* to manipulate them. They allow you to prepare any logical expression you wish. What's the big deal? First, *Boolean Variables* allow you to abandon *Jump to Command*. Everything you could do using *Jump to* you may do easier using *Boolean Variables*. *Jump to* is extremely irritating because it resets itself each time you move *Tactics* around. *Boolean Variables* have no such problems. Second, it is much easier to understand *Tactic Table* that uses *Boolean Variables* (and find mistake if there is any). Third, some logical expression are practically impossible to implement using *Jump to* - it would be too complicated. If you are familiar with boolean logic then names of *Conditions* and *Commands* should be self-explaining. Otherwise, I must ask you to use Google (or Yahoo, or DuckDuckGo etc.) and read a little about the subject; providing tutorial about boolean logic is beyond the scope of this manual. I may suggest to look at: http://math.wikia.com/wiki/Boolean_logic. Below are some rules about *Boolean Variables* in *EMAT*. Look at Figure 10.1 for usage example.

10.2 Rules for Boolean Variables

Each character has his own set of *Boolean Variables* The same as in case of *Saved Enemy* and *Saved Friend*.

At the beginning of combat value of *Boolean Variable* is undefined This means that, at the start of combat, it may be TRUE or FALSE as well. Thus, make sure you store some value in *Boolean Variable* before you use *Condition Logic* to test its value.

The value in *Boolean Variable* is stored until the end of combat This works the same as *Saved Enemy* or *Saved Friend*. Value in *Boolean Variable* changes only if appropriate *Special Command* is used (or combat ends).



Figure 10.1: *Boolean Variables*



11. FAQ

Is this manual complete? No, it is work in progress.

Not all abilities are displayed *FtG UI* fixes this issue: <http://www.nexusmods.com/dragonage/mods/73>

How to enable AI for controlled character? To activate AI for Controlled Characters do the following:

1. Open skills panel
2. Drag *Possessed* icon to QuickBar
3. Click *Possessed* icon on QuickBar

How to enable auto-flanking for rouges? To activate auto-flanking for all rouges do the following:

1. Open skills panel
2. Drag *Traitor* icon to QuickBar
3. Click *Traitor* icon on QuickBar

Endless switch weapon loop You have wrong combinations of *Tactics* and *Behavior*. You are switching weapon in your *Tactic Table* but your chosen *Behavior* switches it back. Look at Figure 4.2. Notice that *Behavior* switches weapon before any of your *Tactic* is evaluated.

What's the difference between AI Status and Affected by spell/talent? *Affected by spell* and *Affected by talent* check whether a creature is affected by **exactly** the given *ability*. Notice that *Paralysis*, *Glyph of Paralysis* and *Mass Paralysis* are three separate abilities. On the other hand, *AI Status* checks whether a creature is under a given *effect*, eg. whether creature is paralyzed, and it does not matter what caused an effect.

How to do Nightmare Combo? Try following:

1. *MultiCondition Setting:Include Sleeping/Rooted:*Add Condition*
2. *MultiCondition Enemy:AI Status:Sleeping:*Store as Saved Enemy*
3. *Saved Enemy:AI Status:Sleeping:Horror*
4. *Enemy:Any:Sleep*

Saved Enemy, like all *Target Types*, will ignore creature that is slept/rooted. You must tell it explicitly that you want to interact with slept/pinned creature. *Saved Enemy* does not know that you stored its content in previous row using *MultiCondition*. It is not smart enough.

How to take advantage of Shattering, but without everyone focusing on the same target?

At present time there is no easy way to do it.

Where is Target Type:Ally? It was replaced by two new *Target Types*:

- *Party Member*
- *Team Member*

Why Target Type:Ally was removed? *Ally* had different meaning in Vanilla game, different in *Advanced Tactics* and it would have different meaning in *EMAT*. I am afraid it would cause much more confusion then replacing it by two new *Target Types*.

What are Ranges values? Ranges values are inherited from Vanilla game. Notice that range Long is useless in practice because at distance of 80.0 meters creature probably is not in combat.

- Short = 3.5 m;
- Medium = 12.0 m;
- Long = 80.0 m
- Bow = 45.0 m

How to set Tactics for summoned creatures? It is not possible.

Is EMAT fully translated for localization? At present time there are only english texts. I don't know how *EMAT* will behave for other languages (english text? no text?). I will upload (someday) Excel file with all texts in order to allow native speakers to prepare translations. But I don't suppose I will make myself to do it in the near future...

What does EMAT version number mean? The meaning is as follows: Major.Minor.Patch If Major number was changed (e.g. from 1.0.0 to 2.0.0) it means that you must rewrite your *Tactic Table*. It may be result of introducing new feature, optimization, bug fixing etc. If Minor number was changed (e.g. from 1.0.0 to 1.1.0) it means that there is full compatibility and that some new features were added. If Patch number is changed (e.g from 1.0.0 to 1.0.1) it means that there is full compatibility and that there are no new features, only bugfixes. This numeration should help you decide whether you want to install a new version immediately or wait with it eg. until you start new game or change party composition.

Where can I find EMAT sourcecode? <https://sourceforge.net/p/dragonage-emat/code/ci/master/tree/>



Index

Symbols

'After you' 23

A

Ability 26
 Ability Mutex 23, 24
 Advanced Tactics 5, 7, 15, 16, 34
 Aggressive 15, 17
 Avoid AoE 15

B

Base Condition 25, 26
 BaseCommand 9
 BaseCondition 9, 27
 Behavior 13–17, 19, 33
 Boolean Variable 31, 32
 Buff 23

C

Can't Attack 23
 Can't Attack AoE 23
 Cautious 15
 Clustered with X enemies 5
 Command 9, 10, 13, 14, 19, 22, 31

Command 26
 compatibility 7
 Condition 9, 10, 15, 19, 20, 22, 23, 25, 27, 31
 Creatures List 10, 12, 21, 26, 27
 Current Target . 10, 12, 16, 17, 19, 21–23, 27

D

Default 15
 Defensive 15
 Dispel 23
 Dispel AoE 24
 Dragon Age 2 25

E

EMAT 5, 7, 13, 14, 16, 23, 31, 34
 Event Manager 7

F

FtG UI 33

G

Game Engine 13

I

installation 7

M

Magic 24
 Modal Abilities 19
 MultiCondition 25–28, 33
 Multicondition 25
 MultiCondition Block 25, 26
 MultiConditon 25
 MultiConditon Block 26

P

Party Member 11, 21, 23
 Party Pool 11
 Passive 15
 Penalty 23
 Possessed 33
 Protective 15

R

Ranged 15

S

Special Command 19, 20, 31
 Spell Combination 21, 22
 Stances 19
 SubCommand 9
 SubCondition 9
 Summoned 10, 11, 29
 Syntax Error 26
 Syntax Rules 25

T

Tactic 9, 10, 13, 14, 19, 23, 25, 31, 33
 Tactic Id 10, 26
 Tactic Owner 9, 19, 21, 25, 27
 Tactic Table 9, 13–16, 20, 23, 31, 33, 34

Target Type 9, 10, 21, 22, 25, 27, 33, 34
 Target Types 25, 28
 Team Member 11, 21
 Team Members 10, 11
 Templar 15
 Thanks and Remarks 5
 Traitor 33

U

Unlimited Variable Storage 5
 Use current condition for next tactic 25